

It runs clean and prints **Total revenue: 498,421.00**. That number is wrong, and nothing in the output tells you so. Your job is to find out why using only breakpoints.

Rule for the exercise: don't read the file. Skim it to know the function names, then go in with the debugger. That's the whole point.

1 - Running the file gives the following output

```
Processed 200 orders
Total revenue: 498,421.00
```

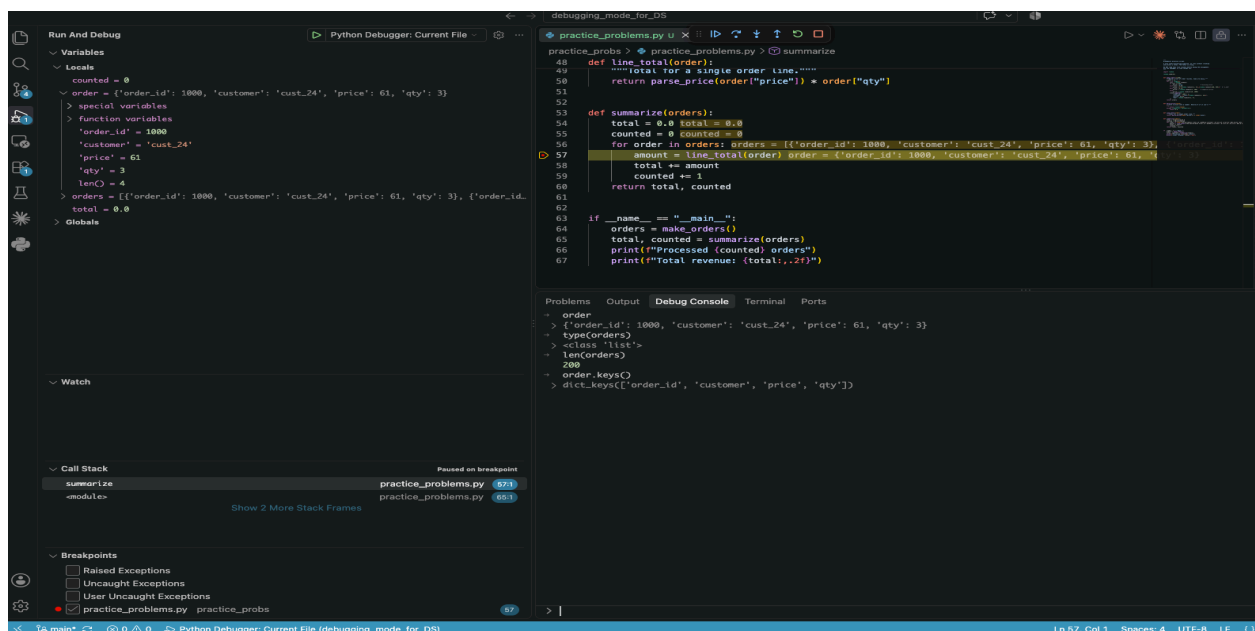
2 - Skimming through the file - Nothing that is too complex - A function that builds the orders and summarizes them into one number

Task 1 — Standard breakpoint

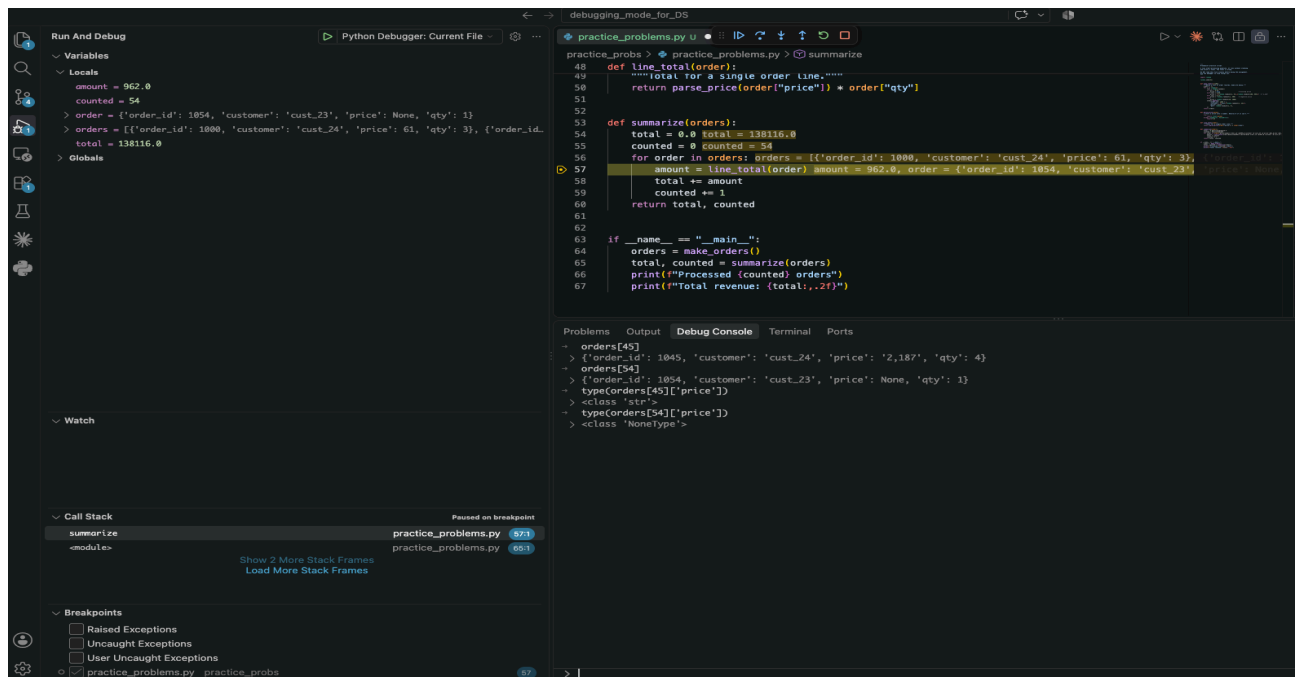
Put one on the `amount = line_total(order)` line in `summarize`. F5.

→ In the Variables pane, what is `order`? What are the four keys? What's `order_id`?

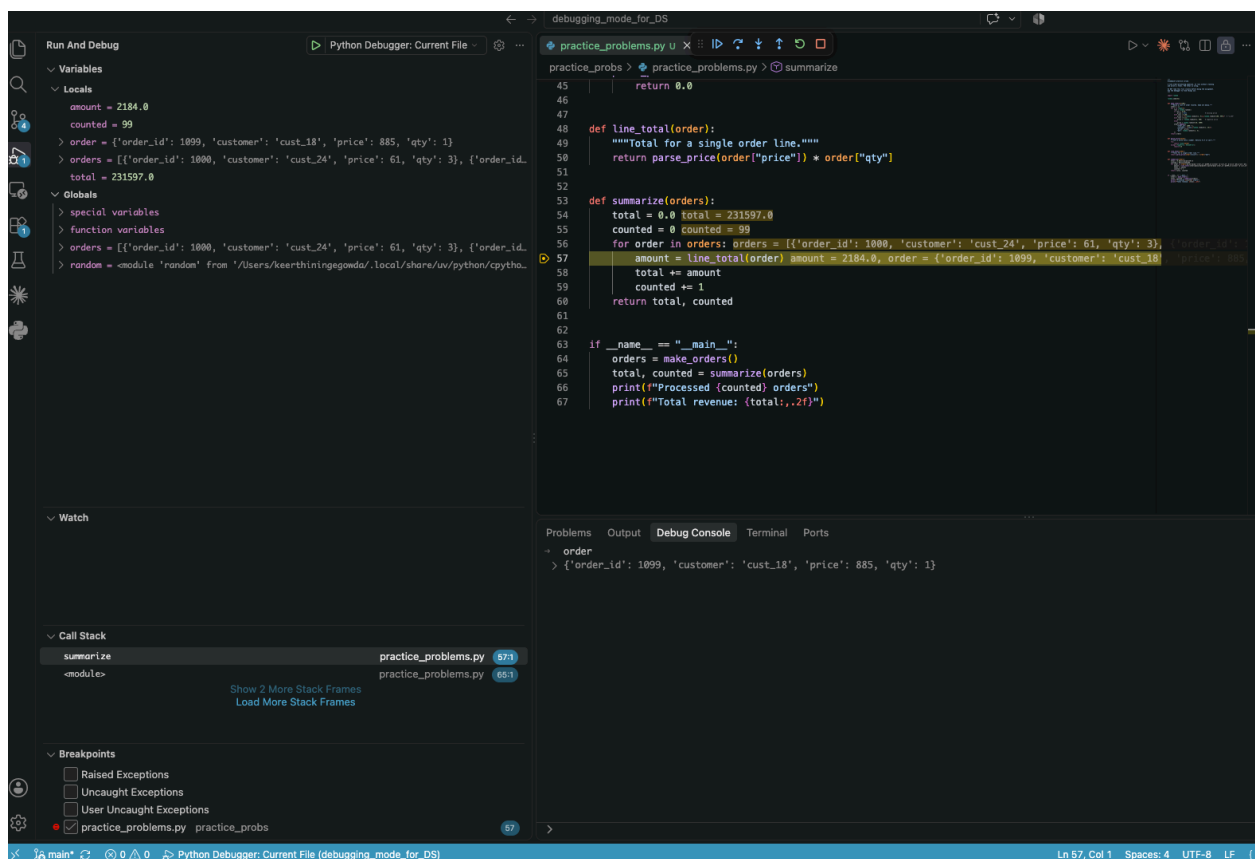
Answer:-



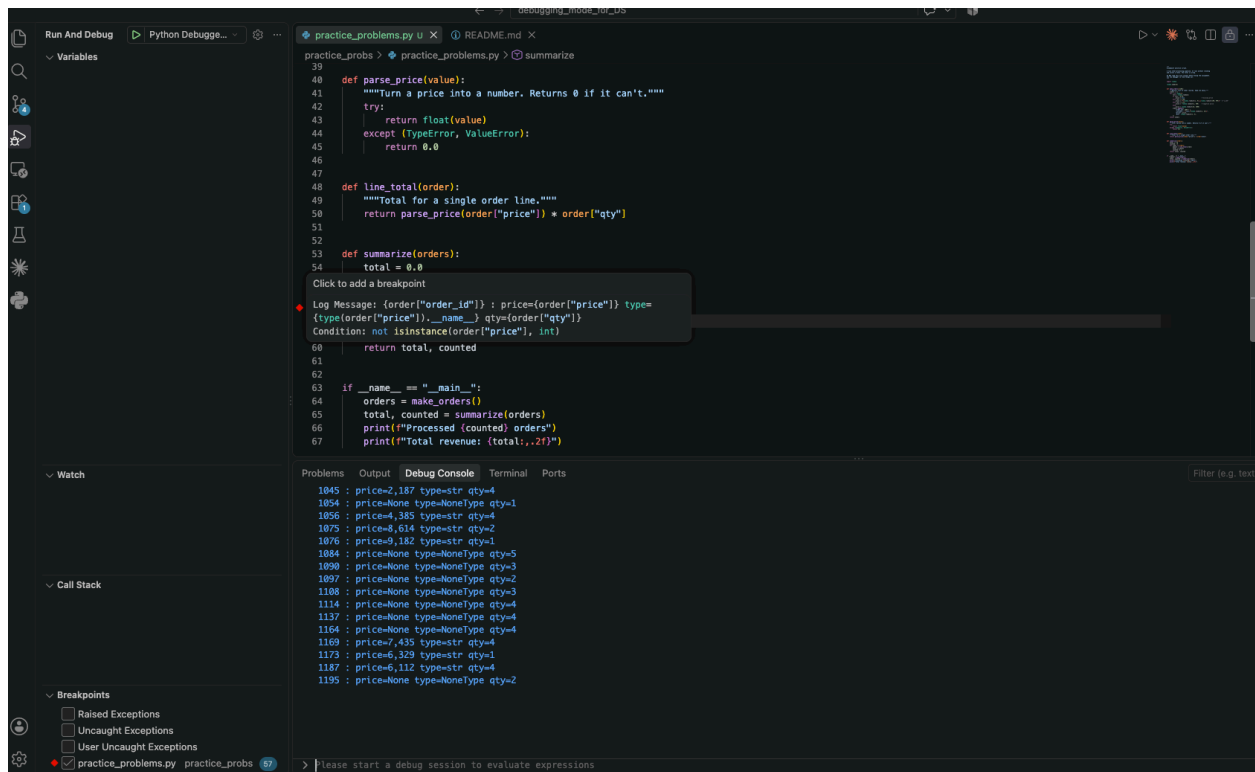
Task 2 - Conditional breakpoint - not isinstance(order["price"], int)



Task 3 - Hit count -



Task 4 - Logpoint and conditionals



```
def parse_price(value):
    """Turn a price into a number. Returns 0 if it can't."""
    try:
        return float(value)
    except (TypeError, ValueError):
        return 0.0

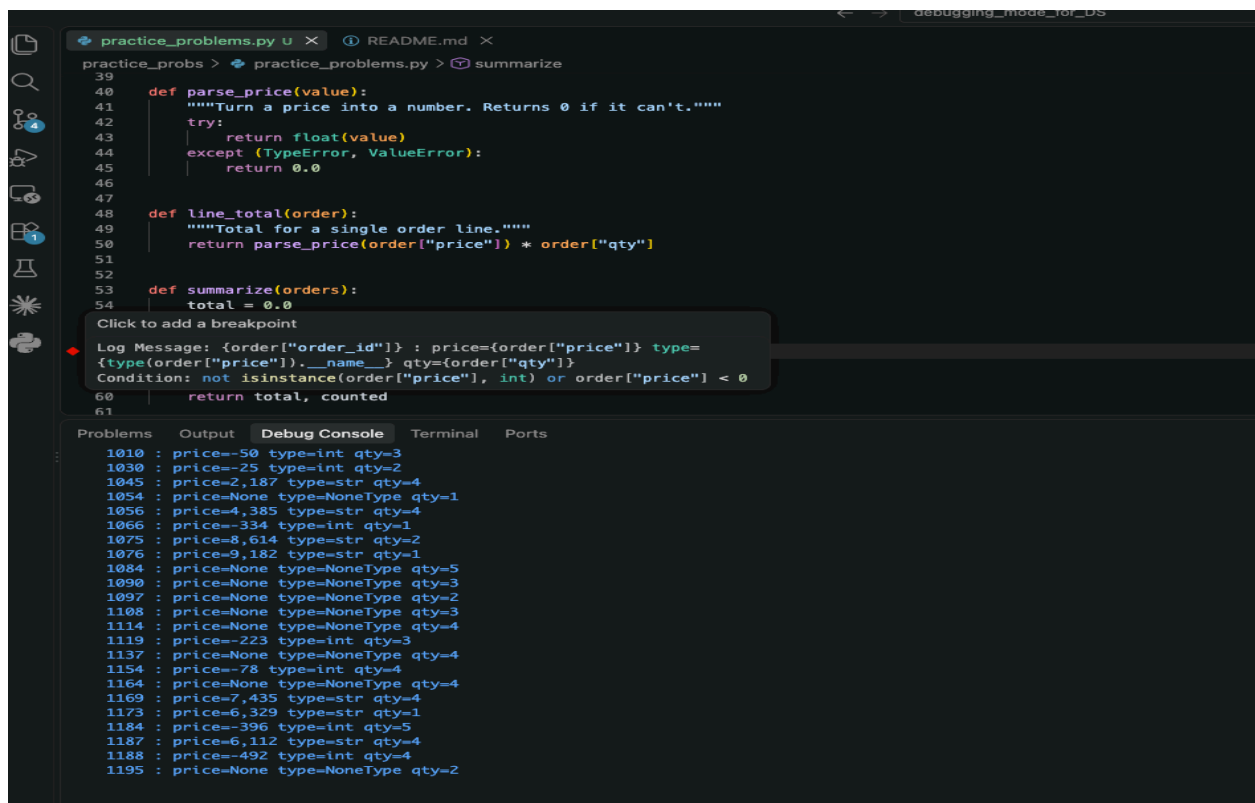
def line_total(order):
    """Total for a single order line."""
    return parse_price(order["price"]) * order["qty"]

def summarize/orders):
    total = 0.0
    for order in orders:
        price = order["price"]
        qty = order["qty"]
        if isinstance(price, int):
            total += line_total(order)
    return total, counted

if __name__ == "__main__":
    orders = make_orders()
    total, counted = summarize(orders)
    print(f"Processed {counted} orders")
    print(f"Total revenue: {total:.2f}")
```

Log Message: {order["order_id"]} : price={order["price"]} type={type(order["price"]).__name__} qty={order["qty"]} Condition: not isinstance(order["price"], int)

1045 : price=2,187 type=str qty=4
1054 : price=None type=NoneType qty=1
1056 : price=4,385 type=str qty=4
1075 : price=8,614 type=str qty=2
1076 : price=9,182 type=str qty=1
1084 : price=None type=NoneType qty=5
1090 : price=None type=NoneType qty=3
1097 : price=None type=NoneType qty=2
1108 : price=None type=NoneType qty=3
1114 : price=None type=NoneType qty=4
1137 : price=None type=NoneType qty=4
1164 : price=None type=NoneType qty=4
1169 : price=7,435 type=str qty=4
1173 : price=6,329 type=str qty=1
1187 : price=6,112 type=str qty=4
1195 : price=None type=NoneType qty=2



```
def parse_price(value):
    """Turn a price into a number. Returns 0 if it can't."""
    try:
        return float(value)
    except (TypeError, ValueError):
        return 0.0

def line_total(order):
    """Total for a single order line."""
    return parse_price(order["price"]) * order["qty"]

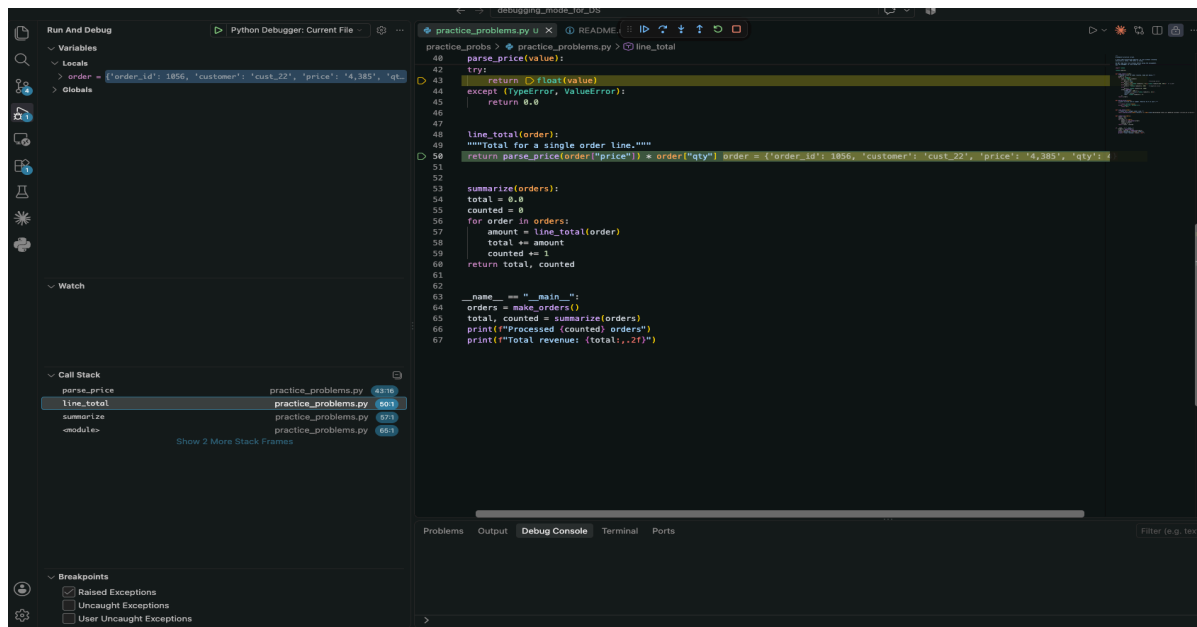
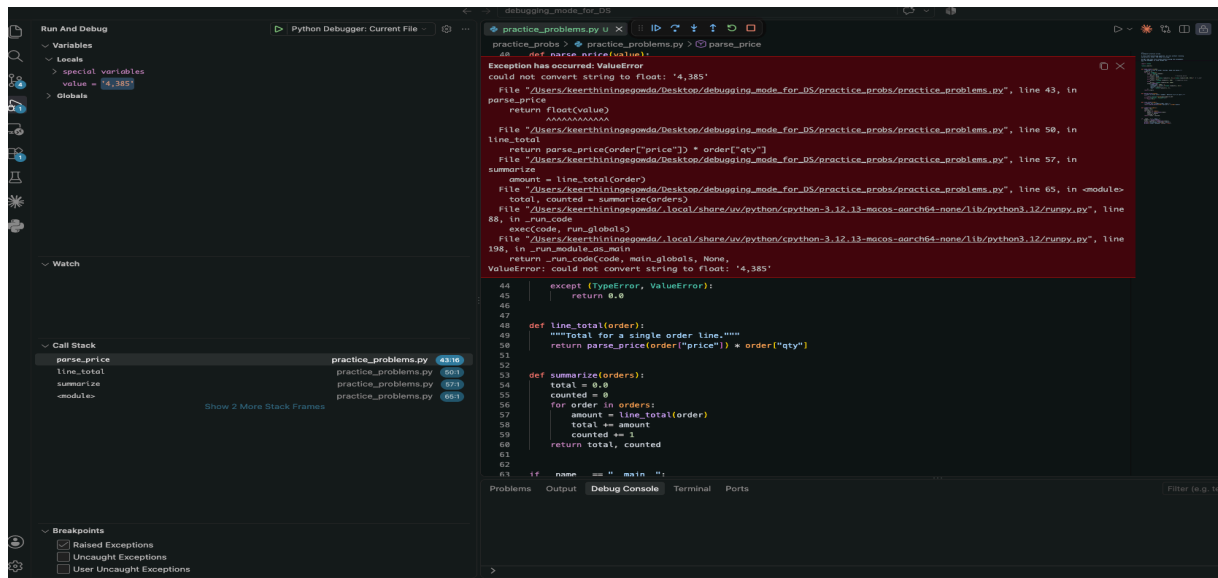
def summarize/orders):
    total = 0.0
    for order in orders:
        price = order["price"]
        qty = order["qty"]
        if isinstance(price, int) and price != 0:
            total += line_total(order)
    return total, counted

if __name__ == "__main__":
    orders = make_orders()
    total, counted = summarize(orders)
    print(f"Processed {counted} orders")
    print(f"Total revenue: {total:.2f}")
```

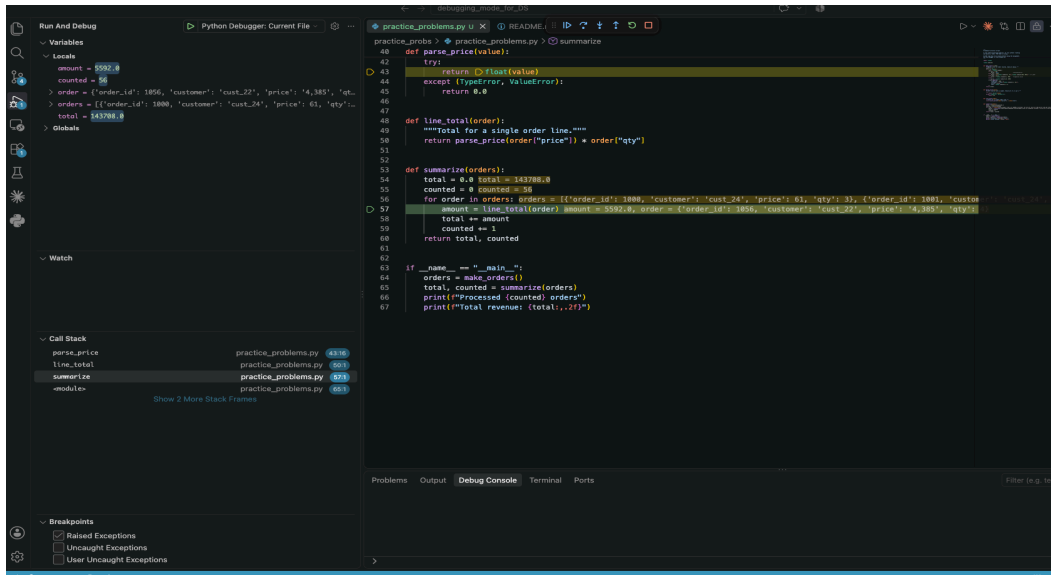
Log Message: {order["order_id"]} : price={order["price"]} type={type(order["price"]).__name__} qty={order["qty"]} Condition: not isinstance(order["price"], int) or order["price"] < 0

1010 : price=-50 type=int qty=3
1030 : price=-25 type=int qty=2
1045 : price=2,187 type=str qty=4
1054 : price=None type=NoneType qty=1
1056 : price=4,385 type=str qty=4
1066 : price=-334 type=int qty=1
1075 : price=8,614 type=str qty=2
1076 : price=9,182 type=str qty=1
1084 : price=None type=NoneType qty=5
1090 : price=None type=NoneType qty=3
1097 : price=None type=NoneType qty=2
1108 : price=None type=NoneType qty=3
1114 : price=None type=NoneType qty=4
1119 : price=-223 type=int qty=3
1137 : price=None type=NoneType qty=4
1154 : price=-78 type=int qty=4
1164 : price=None type=NoneType qty=4
1169 : price=7,435 type=str qty=4
1173 : price=6,329 type=str qty=1
1184 : price=-396 type=int qty=5
1187 : price=6,112 type=str qty=4
1188 : price=-492 type=int qty=4
1195 : price=None type=NoneType qty=2

Task 6 — Exception breakpoint



The amount is from previous stack - In the below image



Key takeaways from this debugging session

- 1) Data has some issues especially as it relates to price - The issue is that the price has mixed types and has also has negative values
- 2) parse_price function squashes valid price values to 0, which will be underestimating total-price
- 3) Sanity checks on order["qty"] was done too. Specifically order["qty"] <= 0 conditional was placed on amount = line_total(order) - The program ran without the trigger of the breakpoint